实验三　可综合时序逻辑电路实验

一．实验目的：

1．　掌握可综合 Verilog 语言进行时序逻辑设计的使用；

2．　学习测试模块的编写、综合和不同层次的仿真。

二．实验内容：

1．设计一款时钟上升沿触发的 D 寄存器,并编写 testbench.

2．设计一款 4bBit 具有并行加载功能的移位寄存器,并编写 testbench

3．设计一款 4 bit 带复位功能的计数器,并编写 testbench 和仿真波形,请包括以下测试点:电路是否能复位? 是否能按预期增加计数? 是否按预定溢出?

4．如何设计一个定时器,(如：时钟频率 20M,定时为 1 秒)并编写 testbench（在第 3 题的基础上完成）

5．串并转换器设计与仿真

三．实验步骤

1．建立工程文件，编写模块源码和测试模块,要求测试模块能对源文件进行比较全面的测试。

2．编译源码和测试模块，用测试模块对源文件进行测试，并综合仿真。得到波形图。

3．记录数据并完成实验报告。

4．使用 QuartusII 软件的 RTL viewer 工具查看/确认所设计的电路。

四．实验报告

1．设计一款时钟上升沿触发的 D 寄存器,并编写 testbench.

模块源码：

```
module d_flip_flop(
        input   wire    clk,
        input   wire    rst,
        input   wire    d,
        output  reg     q
    );
        always @(posedge clk or negedge rst) begin
            if (!rst) begin
```

```
            q <= 1'b0;
        end else begin
            q <= d;
        end
    end
endmodule
```

测试模块：

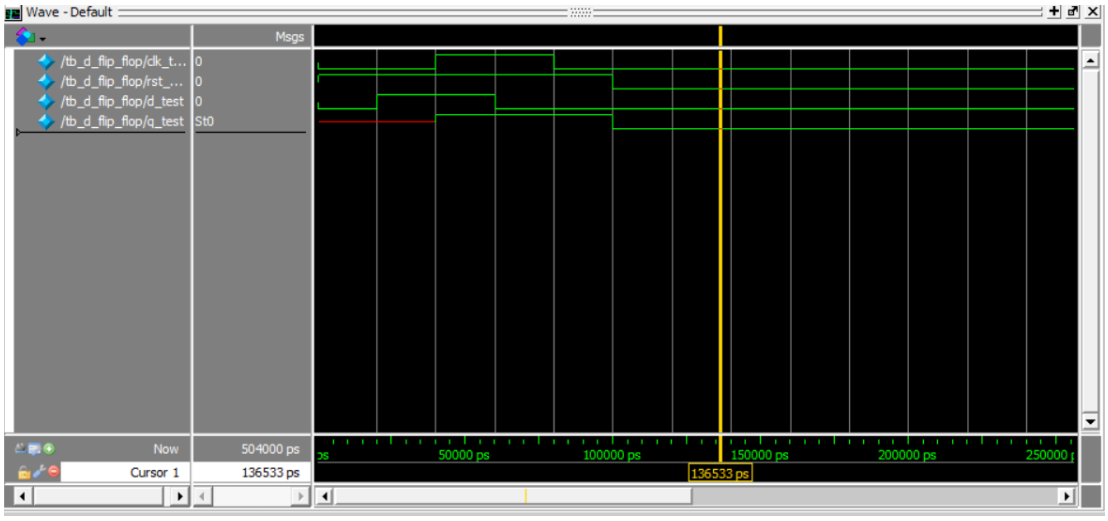
```
`timescale 1ns/1ps
module tb_d_flip_flop();
    reg     clk_test;
    reg     rst_test;
    reg     d_test;
    wire    q_test;

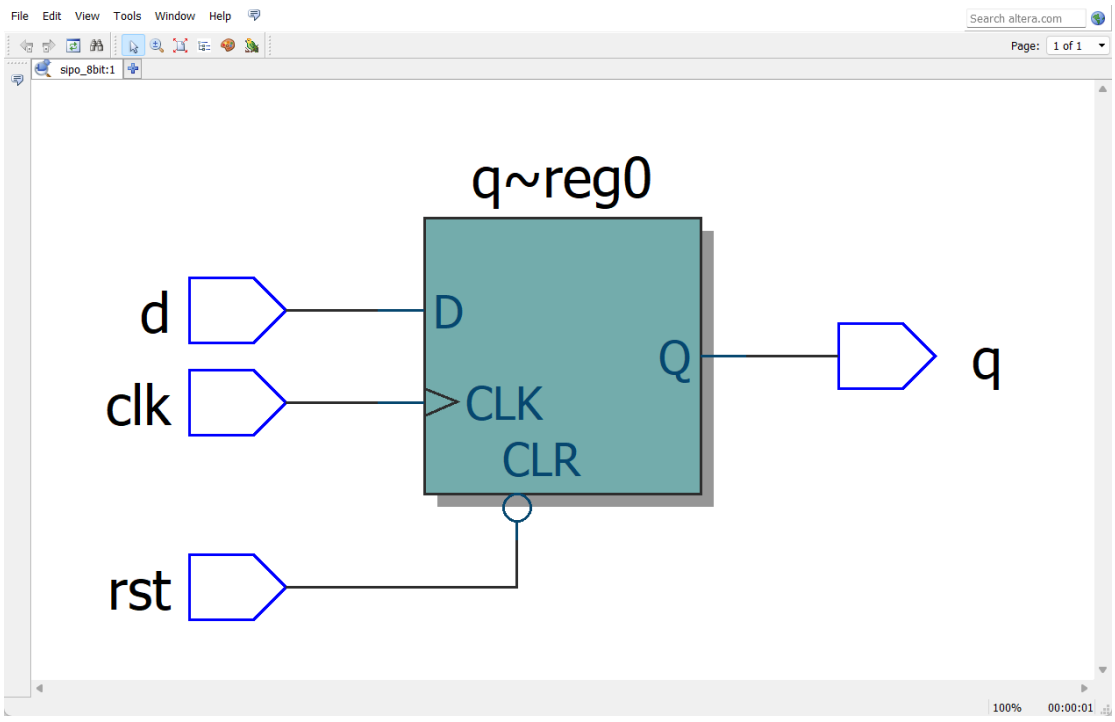
    initial begin
        clk_test = 1'b0;
        rst_test = 1'b1;
        d_test   = 1'b0;
    end

    initial begin
        #20     {clk_test, d_test} = 2'b01;
        #20     {clk_test, d_test} = 2'b11;
        #20     {clk_test, d_test} = 2'b10;
        #20     {clk_test, d_test} = 2'b00;
        #20     rst_test = 1'b0;
    end

    d_flip_flop test(.clk(clk_test),
                                        .rst(rst_test),
                                        .d(d_test),
                                        .q(q_test)
    );
Endmodule
```

运行后可得到波形图以及 RTL 图形：





2．设计一款 4bBit 具有并行加载功能的移位寄存器，并编写 testbench

模块源码：module shift_register_4bit_parallel(

```
        input    wire    clk,
        input    wire    rst_n,
        input    wire    load,
        input    wire    shift_en,
        input    wire    serial_in,
        input    wire    [3:0]    data_in,
        output   reg     [3:0]    date_out,
        output   wire    serial_out
);
        reg     [3:0]    shift_reg;
```

```
    always @(posedge clk or negedge rst_n) begin
        if (!rst_n) begin
            shift_reg <= 4'b0000;
        end else if (load) begin
            shift_reg <= data_in;
        end else if (shift_en) begin
            shift_reg <= {shift_reg[2:0], serial_in};
        end
    end

    assign serial_out = shift_reg[3];
    assign data_out = shift_reg;
endmodule
```

测试模块：

```
`timescale 1ns/1ps
module shift_register_4bit_parallel(
    input       wire        clk,
    input       wire        rst_n,
    input       wire        load,
    input       wire        shift_en,
    input       wire        serial_in,
    input       wire        [3:0]       data_in,
    output      reg         [3:0]       data_out,
    output      wire        serial_out
);

    always @(posedge clk or negedge rst_n) begin
        if (!rst_n) begin
            data_out <= 4'b0000;
        end else if (load) begin
            data_out <= data_in;
        end else if (shift_en) begin
            data_out <= {data_out[2:0], serial_in};
        end
    end

    assign serial_out = data_out[3];
endmodule


module tb_shift_register_4bit_parallel();
    reg             clk_test;
    reg             rst_n_test;
    reg             load_test;
```

```
reg             shift_en_test;
reg             serial_in_test;
reg   [3:0] data_in_test;
wire [3:0] data_out_test;
wire            serial_out_test;

initial begin
    clk_test = 1'b0;
end

always begin
    #20 clk_test = ~clk_test;
end

initial begin
    rst_n_test = 1'b1;
    #5    rst_n_test = 1'b0;
    #10 rst_n_test = 1'b1;
end

initial begin
    load_test          = 1'b0;
    shift_en_test    = 1'b1;
    data_in_test     = 4'b0000;
end

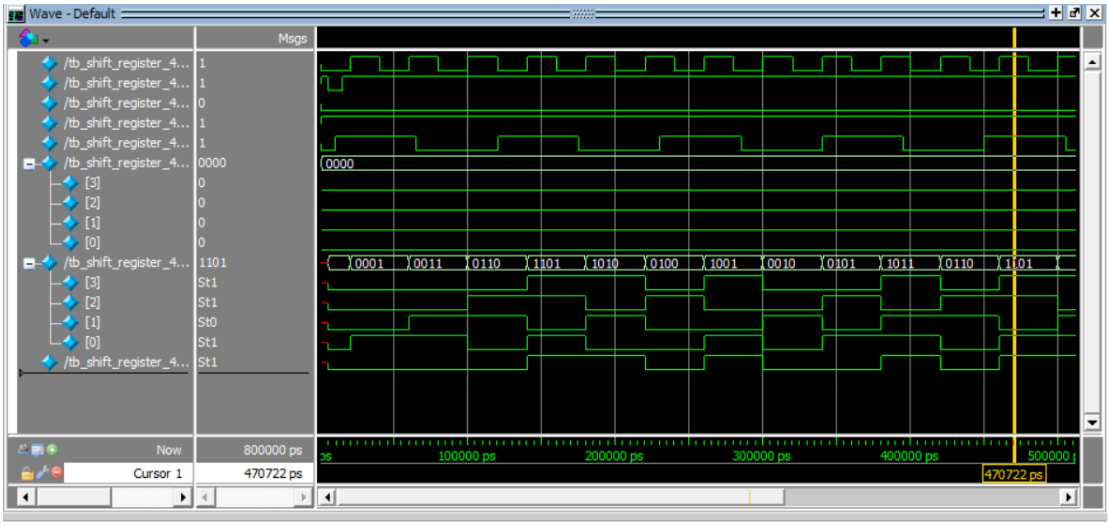
initial begin
    serial_in_test = 1'b0;
    #10;
    forever begin
        serial_in_test = ~serial_in_test;
        #55;
    end
end

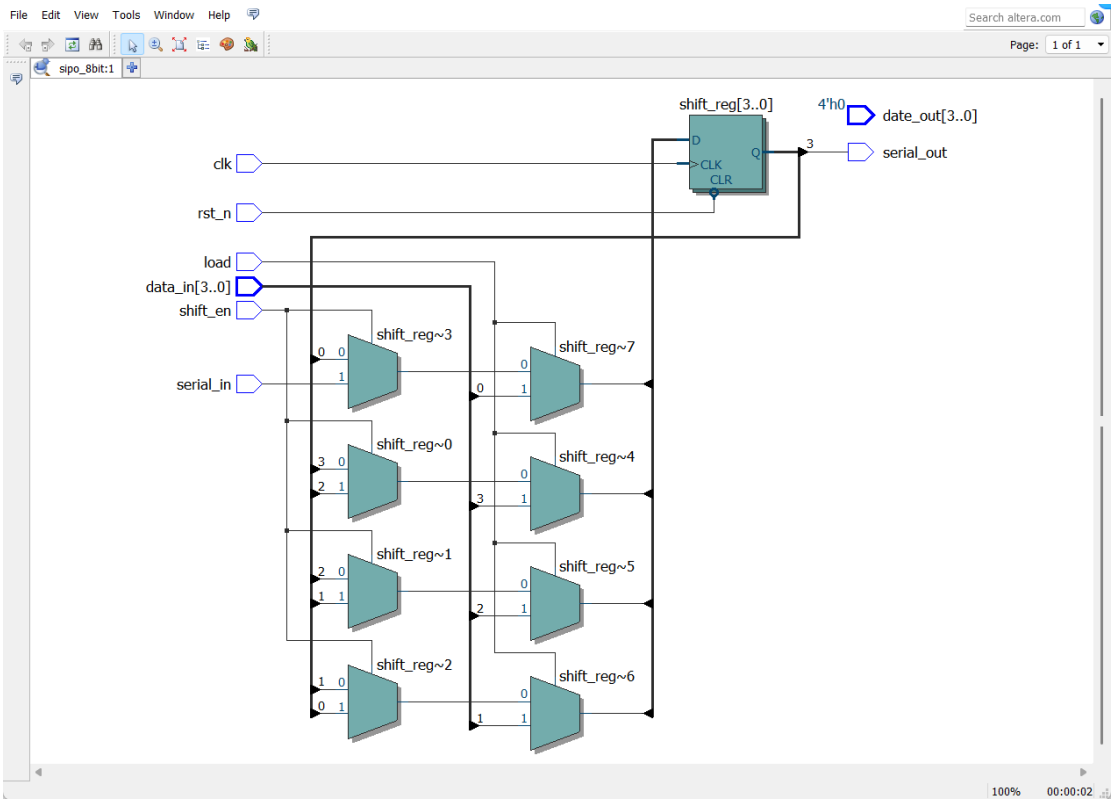
shift_register_4bit_parallel test(
    .clk(clk_test),
    .rst_n(rst_n_test),
    .load(load_test),
    .shift_en(shift_en_test),
    .serial_in(serial_in_test),
    .data_in(data_in_test),
    .data_out(data_out_test),
    .serial_out(serial_out_test)
```

);

Endmodule

运行后可得到波形图以及 RTL 图形：





3．设计一款 4 bit 带复位功能的计数器，并编写 testbench 和仿真波形，请包括以下测试点：电路是否能复位？是否能按预期增加计数？是否按预定溢出？

模块源码：module sync_counter_4bit (

input wire clk,

input wire rst_n,

```
        output reg [3:0] count
    );

    always @(posedge clk) begin
        if (!rst_n) begin
            count <= 4'b0000;
        end else begin
            count <= count + 1'b1;
        end
    end

    endmodule
```

测试模块：

```
`timescale 1ns/1ps
  module tb_sync_counter_4bit ();
        reg clk_test;
        reg rst_n_test;
        wire [3:0] count_test;

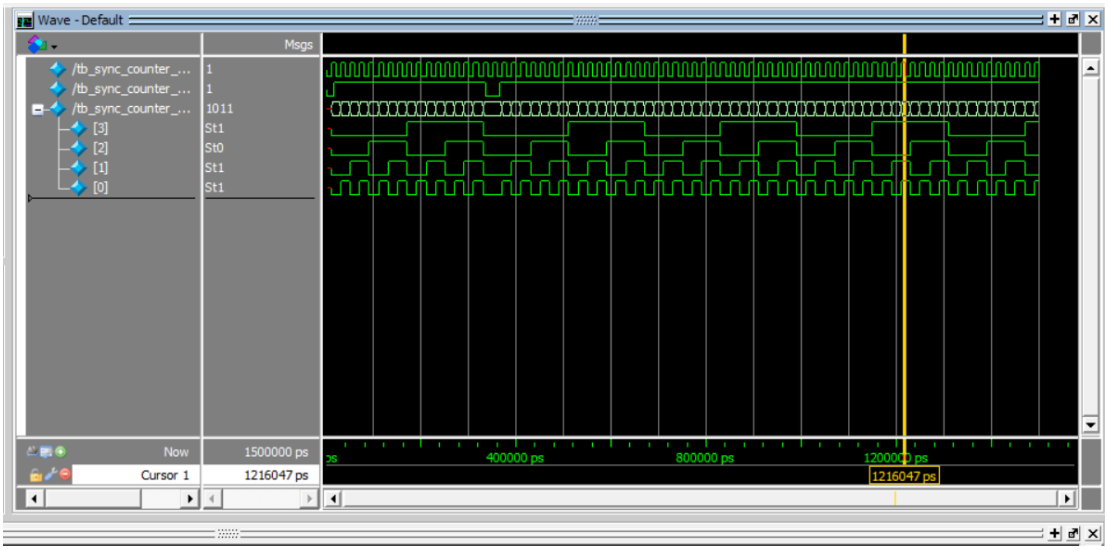
        initial begin
            clk_test = 0;
        end

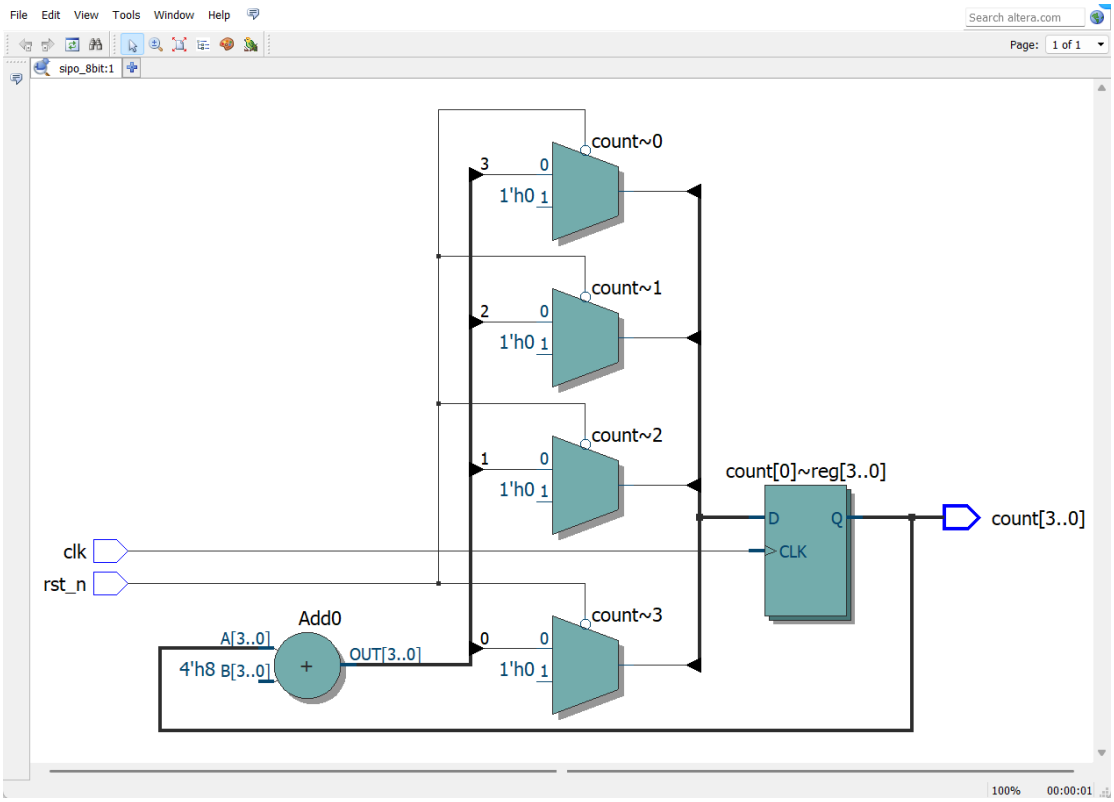
        initial begin
            forever begin
                #10 clk_test = ~clk_test;
            end
        end

        initial begin
            rst_n_test = 0;
            #15 rst_n_test = 1;
            #320 rst_n_test = 0;
            #30 rst_n_test = 1;
        end

        sync_counter_4bit test(
            .clk(clk_test),
            .rst_n(rst_n_test),
            .count(count_test)
        );
    Endmodule
```

运行后可得到波形图以及 RTL 图形：





4．如何设计一个定时器,(如：时钟频率 20M,定时为 1 秒)并编写 testbench（在第 3 题的基础上完成）

模块源码：module sync_counter_4bit (

input wire clk,

input wire rst_n,

output reg [3:0] count

);

always @(posedge clk) begin

```
        if (!rst_n) begin
            count <= 4'b0000;
        end else begin
            count <= count + 1'b1;
        end
    end

    endmodule



    module timer(
        input wire clk,
        input wire rst_n,
        input wire [3:0] target,
        output wire [3:0] count,
        output wire match
    );

        assign match = (target == count)? 1 : 0;

        sync_counter_4bit counter(
            .clk(clk),
            .rst_n(rst_n),
            .count(count)
        );
    Endmodule
```

测试模块：

```
`timescale 1ns/1ps
  module tb_timer ();
      reg clk_test;
      reg rst_n_test;
      reg [3:0] target_test;
      wire [3:0] count_test;
      wire match_test;

      initial begin
          clk_test = 0;
          forever begin
              #10 clk_test = ~clk_test;
          end
      end

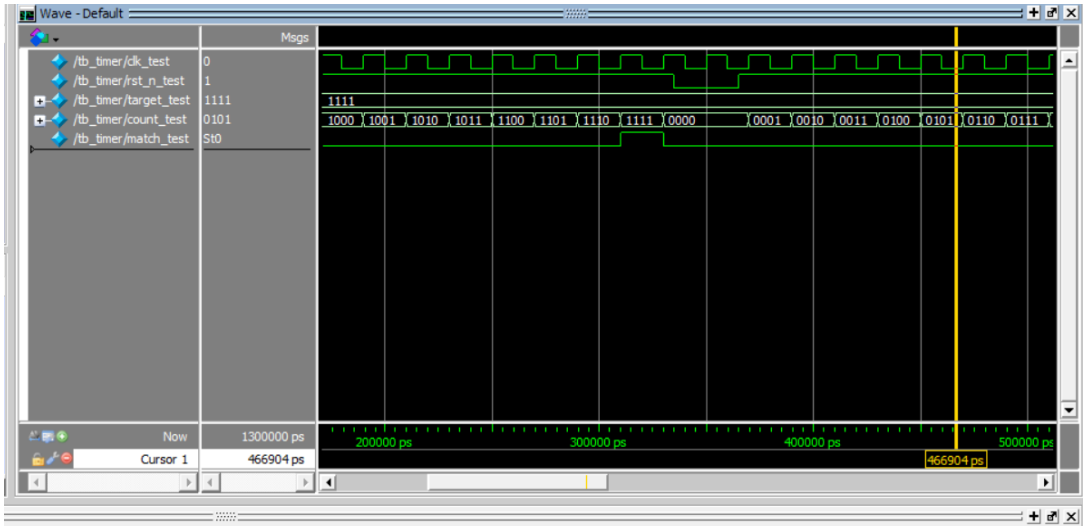
      initial begin
```

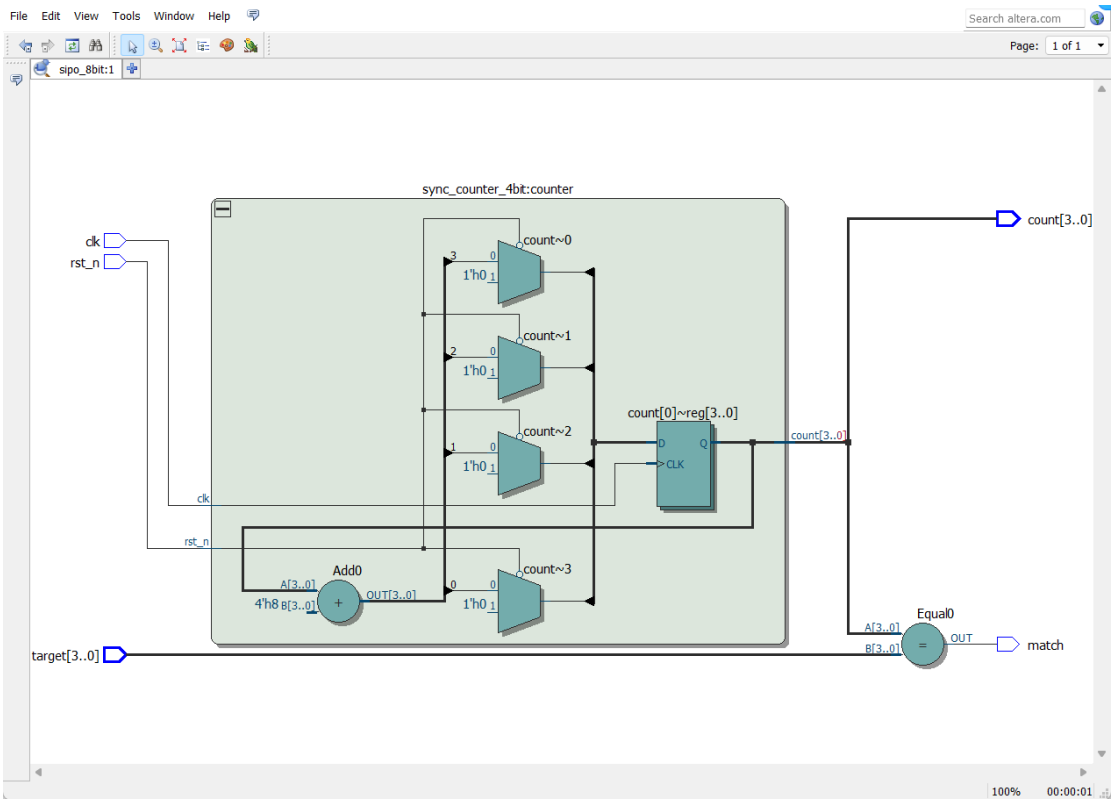
```
        target_test = 4'b1111;
    end

    initial begin
        rst_n_test = 0;
        #15 rst_n_test = 1;
        #320 rst_n_test = 0;
        #30 rst_n_test = 1;
    end

    timer test(
        .clk(clk_test),
        .rst_n(rst_n_test),
        .count(count_test),
        .target(target_test),
        .match(match_test)
    );
Endmodule
```

运行后可得到波形图以及 RTL 图形：





5．串并转换器设计与仿真

模块源码：

```
module sipo_8bit (
        input clk,          // 时钟
        input rst_n,
        input s_in,         // 串行数据输入
        output [7:0] p_out // 并行数据输出
);
reg [7:0] shift_reg;
assign p_out = shift_reg;
always @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
        shift_reg <= 8'h00;
    end
    else begin
        shift_reg <= {s_in, shift_reg[7:1]};
    end
end
endmodule
```

测试模块：

```
`timescale 1ns/1ps
  module sipo_8bit_tb;
  reg clk;
  reg rst_n;
```

```
reg s_in;
wire [7:0] p_out;

parameter CLK_PERIOD = 10; //  时钟周期  10ns

sipo_8bit DUT (
        .clk(clk),
        .rst_n(rst_n),
        .s_in(s_in),
        .p_out(p_out)
);
initial begin
        clk = 0;
        forever #(CLK_PERIOD/2) clk = ~clk; // 5ns  翻转一次，周期  10ns
end

initial begin

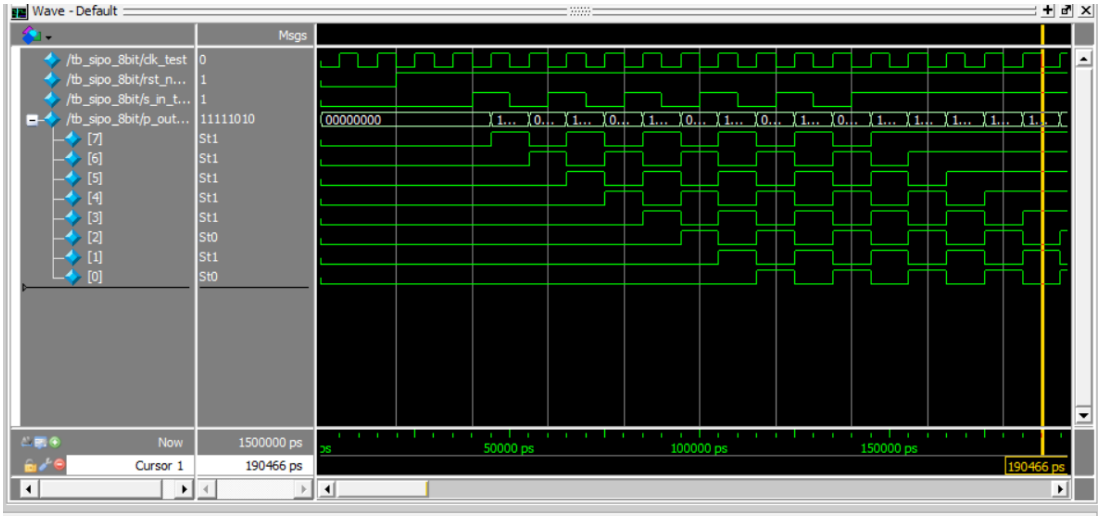
        s_in = 0;
        rst_n = 0;
        # (CLK_PERIOD * 2); //  保持复位  2  个时钟周期
        rst_n = 1;
        #CLK_PERIOD; s_in = 0;
        #CLK_PERIOD; s_in = 0;
        #CLK_PERIOD; s_in = 1;
        #CLK_PERIOD; s_in = 0;
        #CLK_PERIOD; s_in = 1;
        #CLK_PERIOD; s_in = 1;
        #CLK_PERIOD; s_in = 0;
        #CLK_PERIOD; s_in = 1;

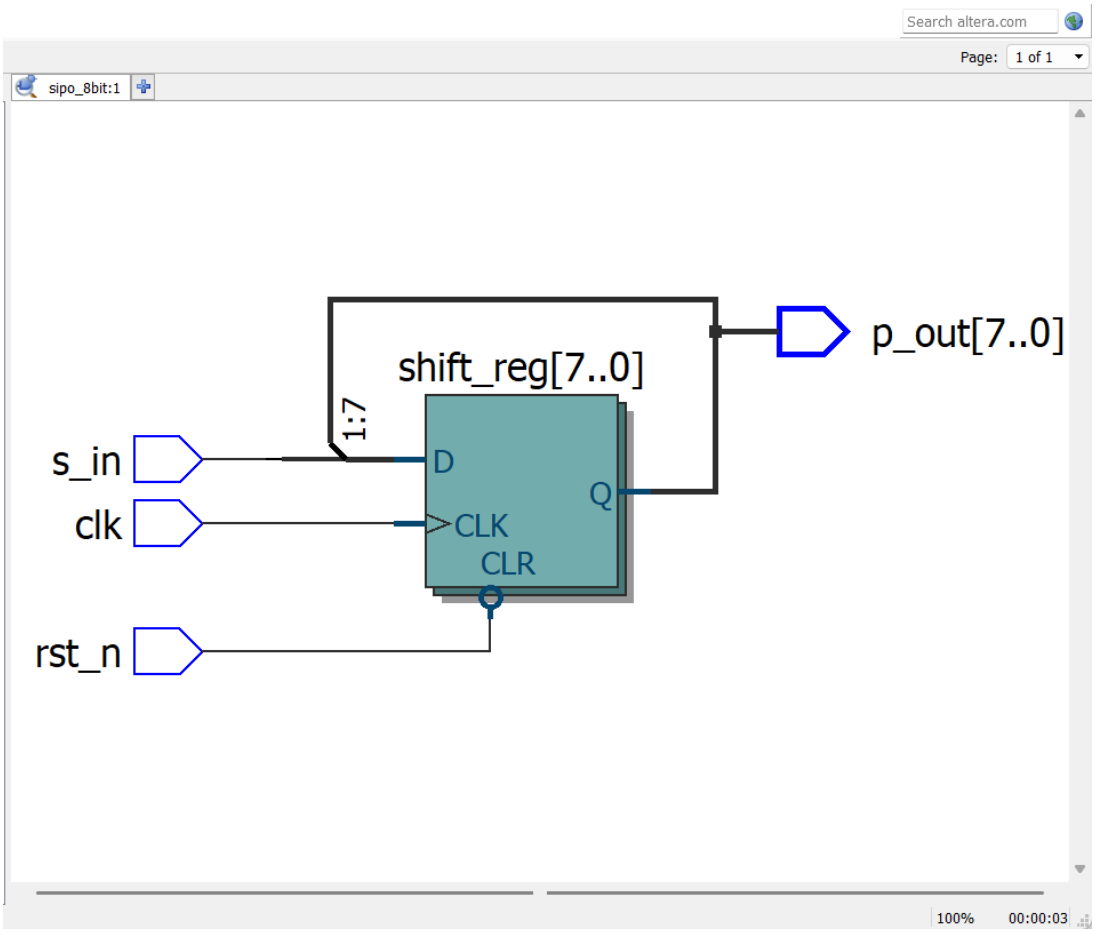
        # (CLK_PERIOD * 5);

        $finish; //  结束仿真
end

endmodule
```

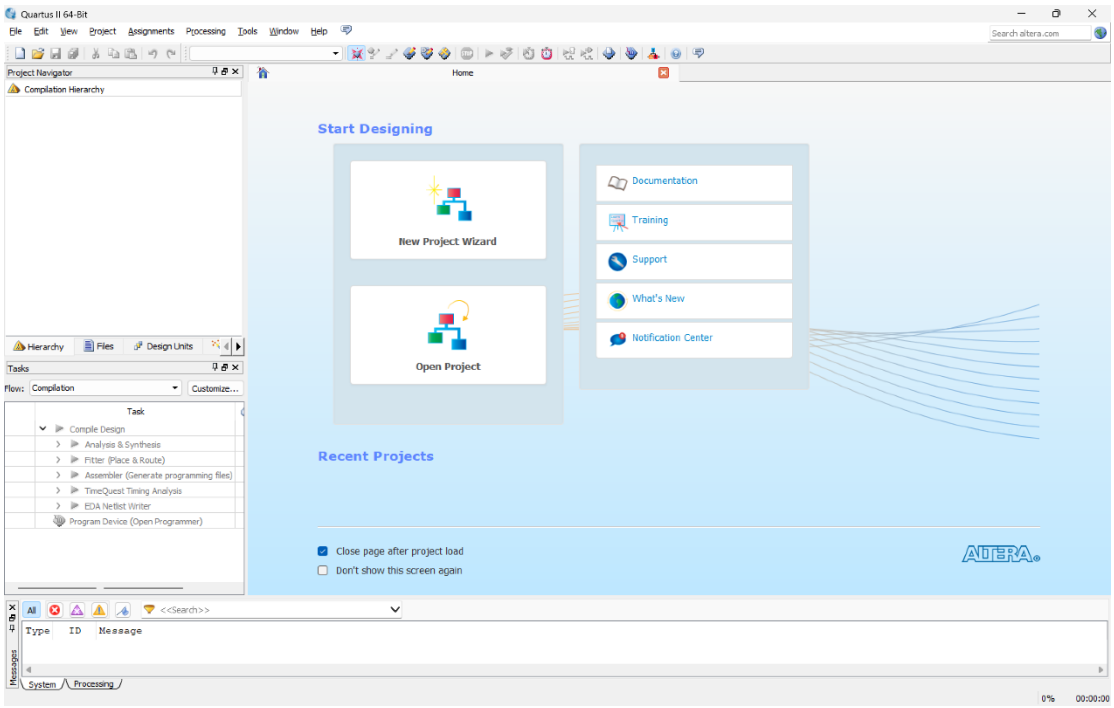
运行后可得到波形图以及 RTL 图形：



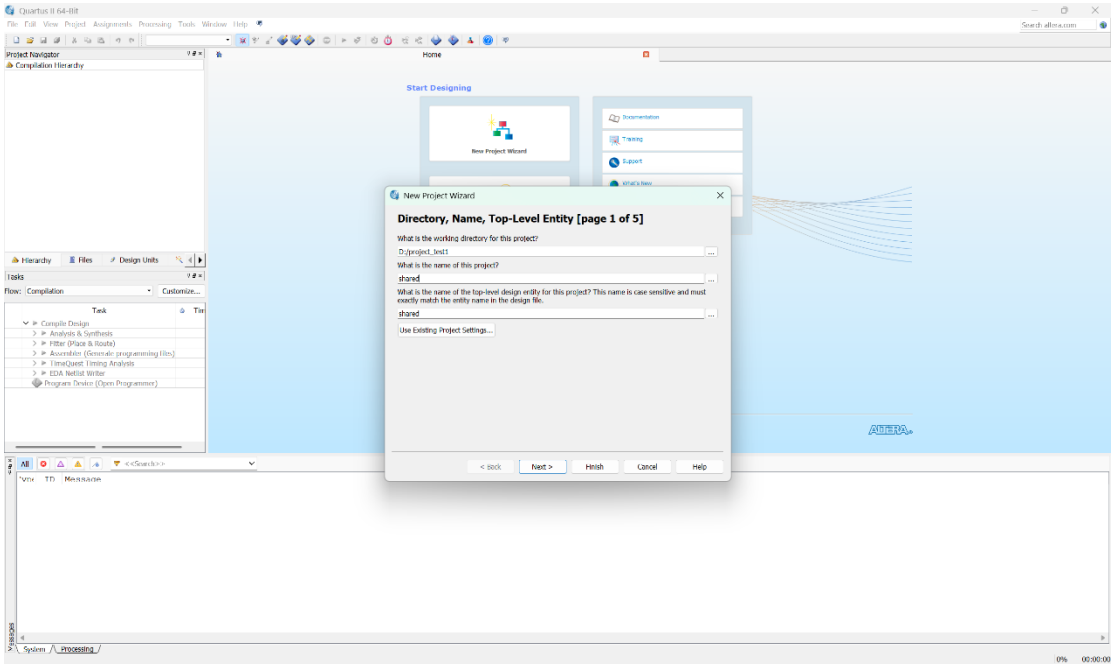


五. QuartusII 软件基本使用步骤

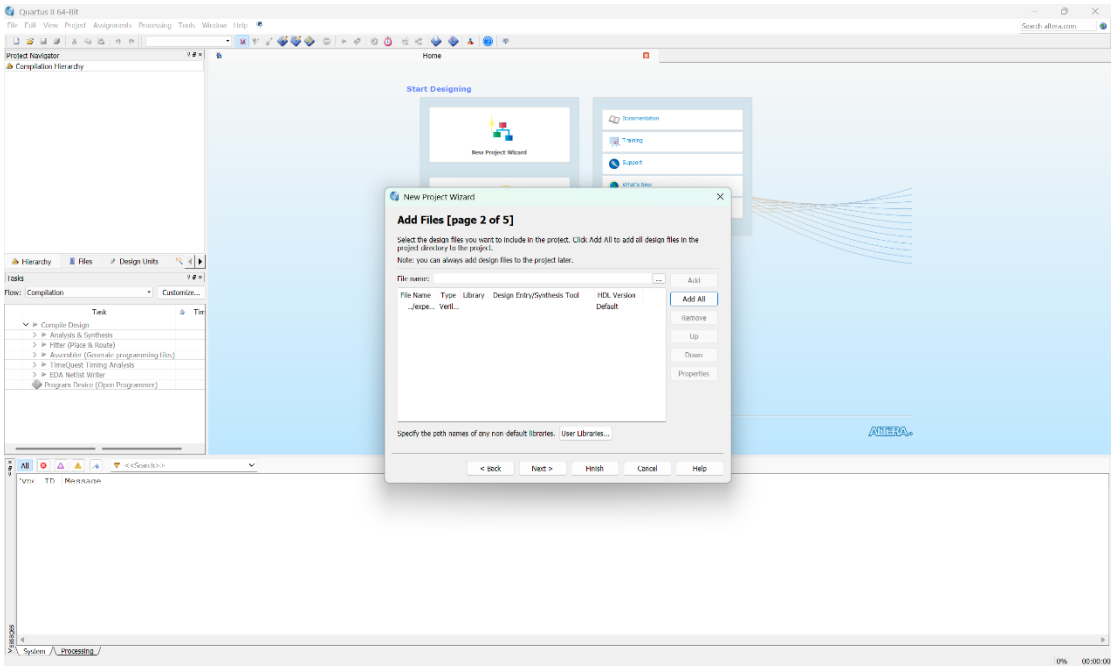
1. 启动 QuatusII 软件如下:



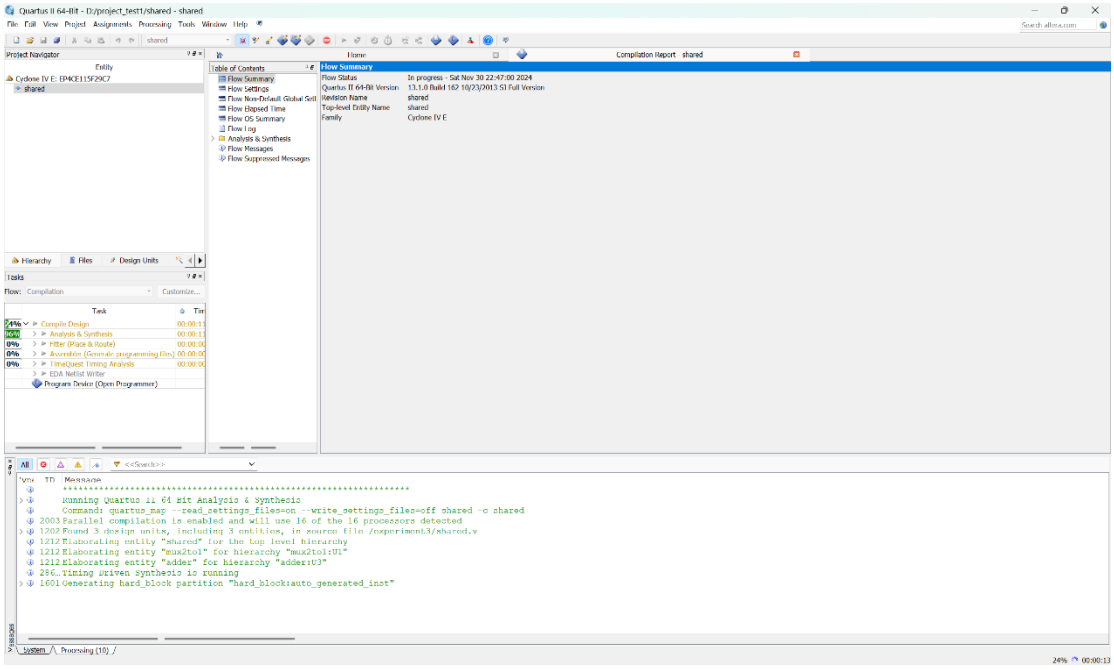
2. 新建工程 New project (注意工程名和设计文件的 module 名保持一致),选择和开发板一致的 FPGA 器件型号(本课程为 EP4CE115F29C7)：



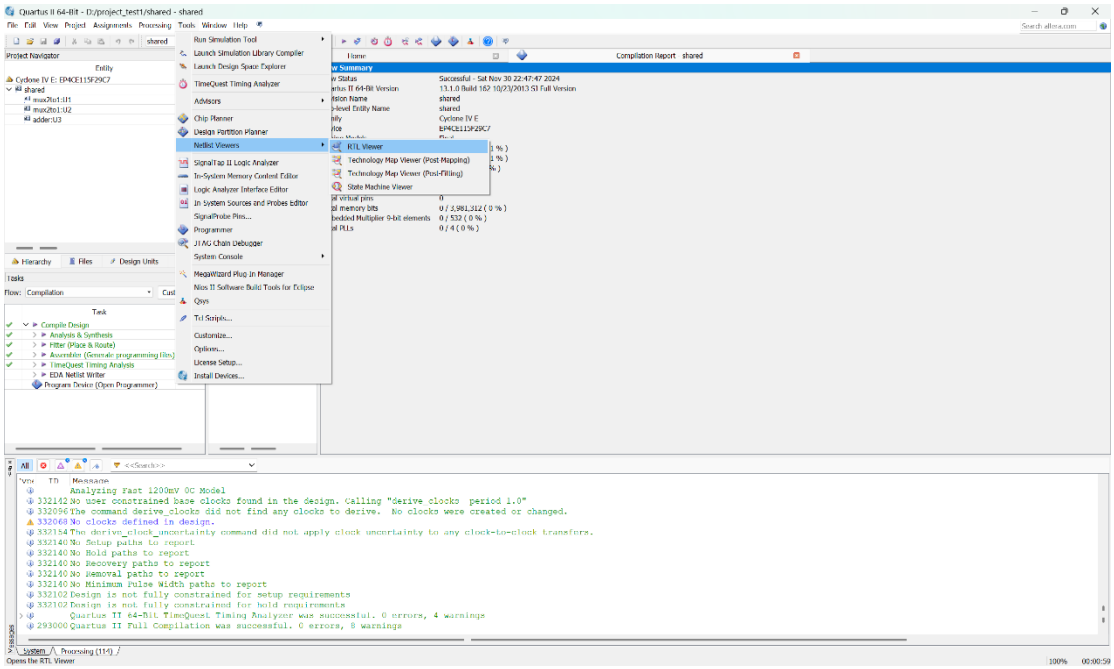
3. Add to project,将全部源文件添加到工程中：

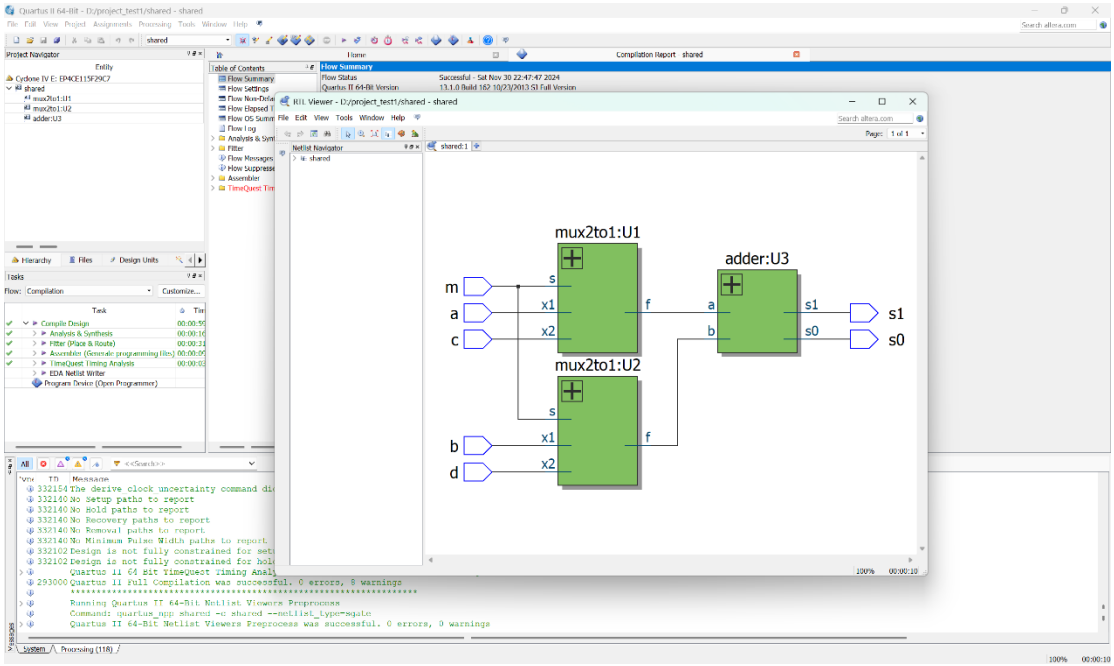


4. Start Compilation ,编译源文件 (如有错误修改后,重新编译)



5. 使用 Tool->RTL viewer 工具查看电路图结构,是否和预期设计一致.





六．实验收获与心得

本次实验让我对 Verilog HDL 的可综合设计有了深入理解。我掌握了使用语法来描述 D 寄存器、移位寄存器、计数器和串并转换器等核心时序逻辑单元。通过定时器设计，我实践了模块化和层次化的设计方法，提高了代码的复用性。编写 Testbench 并观察波形图是验证时序逻辑的关键步骤，它直观地确认了电路的复位、计数溢出和数据移位等功能是否满足预期的时序要求。更重要的是，通过 Quartus II 的 RTL Viewer 工具，我将 Verilog 代码映射到了实际的硬件结构上，这帮助我理解了可综合代码必须能转换为实际的寄存器和组合

逻辑。本次实验成功地将理论知识与 EDA 工具实践相结合，为后续的复杂数字系统设计打下了坚实的基础。